

Objetivo

O objetivo desta atividade prática em laboratório é implementar uma **árvore N-ária** utilizando os conceitos de programação orientada a objetos. A árvore N-ária permite que cada nó possua qualquer quantidade de filhos e será representada pela técnica do **primeiro filho e próximo irmão**.

Além disso, a estrutura implementada deverá ser reutilizada em uma aplicação prática: a representação de uma **estrutura organizacional simplificada do IFC**.

Parte 1 – Implementação da Árvore N-ária

1. Inicialmente crie os seguintes arquivos fonte:

- **NoArvoreNaria**: classe que representa um nó da árvore;
- **ArvoreNaria**: classe que representa a árvore N-ária;
- **ArvoreMain**: classe com o método principal para demonstrar o funcionamento da árvore e da aplicação prática.

2. Estrutura sugerida para a classe **NoArvoreNaria**:

```
public class NoArvoreNaria {  
    private String info;  
    private NoArvoreNaria prim;  
    private NoArvoreNaria prox;  
  
    public NoArvoreNaria(String info) {  
        this.info = info;  
        this.prim = null;  
        this.prox = null;  
    }  
  
    // getters e setters  
}
```

3. Significado dos atributos:

- **info**: string armazenada no nó;
- **prim**: referência para o primeiro filho do nó;
- **prox**: referência para o próximo irmão do nó.

4. A classe `ArvoreNaria` deve possuir, no mínimo, um atributo para armazenar a referência à raiz da árvore:

```
private NoArvoreNaria raiz;
```

5. Métodos a serem implementados na classe `ArvoreNaria`:

- (a) `public ArvoreNaria()`: construtor que instancia uma árvore vazia;
- (b) `public void defineRaiz(NoArvoreNaria r)`: define o nó raiz principal da árvore;
- (c) `public void insereFilho(NoArvoreNaria pai, NoArvoreNaria filho)`: insere o nó filho como filho do nó pai;
- (d) `public boolean vazia()`: retorna `true` se a árvore estiver vazia, ou `false` caso contrário;
- (e) `public boolean pertence(String v)`: verifica se a string `v` está armazenada em algum nó da árvore;
- (f) `public int numNos()`: retorna a quantidade total de nós da árvore;
- (g) `public int folhas()`: retorna a quantidade de nós do tipo folha;
- (h) `public int altura()`: retorna a altura da árvore;
- (i) `public boolean igual(ArvoreNaria a)`: verifica se a árvore atual e a árvore `a` possuem exatamente a mesma estrutura e os mesmos valores;
- (j) `public String toString()`: retorna uma representação textual da árvore em formato hierárquico.

6. A árvore desta atividade é uma **árvore N-ária geral**. Portanto, a montagem da estrutura deve ser feita explicitamente por meio da criação dos nós e da ligação entre filhos e irmãos.

7. A operação `insereFilho` deve inserir uma subárvore como filha de um nó previamente criado. Uma estratégia simples é inserir o novo filho no início da lista de filhos do pai:

```
filho.setProx(pai.getPrim());  
pai.setPrim(filho);
```

8. Um nó folha é aquele cuja referência `prim` é `null`, isto é, ele não possui filhos.
9. Sempre que necessário, implemente métodos privados auxiliares recursivos para percorrer os filhos e irmãos de cada nó.

Parte 2 – Problema Aplicado: Estrutura Organizacional do IFC

Após implementar a árvore N-ária, utilize-a para representar uma **estrutura organizacional simplificada do Instituto Federal Catarinense**. Nesta modelagem, cada nó pode representar uma instituição, campus, curso, disciplina, setor ou coordenação. Como a árvore desta atividade armazena **strings**, utilize diretamente os nomes dos elementos representados.

Implemente, na classe `ArvoreMain`, as seguintes funcionalidades:

- 1. Montar manualmente uma árvore N-ária com pelo menos 12 nós;
- 2. Utilizar a raiz "IFC";
- 3. Criar pelo menos três nós filhos da raiz representando campi;

4. Criar pelo menos dois níveis abaixo de algum campus, por exemplo cursos e disciplinas;
5. Imprimir a árvore em formato textual hierárquico;
6. Informar o número total de nós da árvore;
7. Informar a quantidade de folhas da árvore;
8. Informar a altura da árvore;
9. Verificar se determinados nomes pertencem à árvore;
10. Criar uma segunda árvore e testar se as duas estruturas são iguais;
11. Alterar a segunda árvore e testar novamente se as duas estruturas continuam iguais.

Exemplo de contexto:

```
"IFC"  
"Campus Blumenau"  
"Campus Camboriu"  
"Campus Araquari"  
"BCC"  
"Pedagogia"  
"Administracao"  
"Estruturas de Dados I"  
"Programacao II"  
"Banco de Dados"  
"Coordenacao de Ensino"  
"Biblioteca"
```

Nesse exemplo, a raiz representa a instituição, os filhos da raiz representam campi, e os níveis inferiores representam cursos, disciplinas e setores.

Observações

- A implementação deve representar uma árvore N-ária geral, sem aplicar regras de ordenação;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos;
- Tenha cuidado ao percorrer a árvore: cada nó pode ter uma lista de filhos, e cada filho pode ter uma lista de irmãos;
- Sempre que necessário, implemente métodos privados auxiliares recursivos para realizar as operações sobre a árvore;
- Implemente na classe `ArvoreMain` um exemplo completo demonstrando o funcionamento da árvore e da aplicação prática.

Observação Final

Após implementar todas as classes e métodos, desenvolva um programa principal que demonstre claramente o funcionamento completo da árvore N-ária e da estrutura organizacional modelada sobre ela.